САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

Дисциплина: Бэк-энд разработка

Отчет

Лабораторная работа №5
«Настройка конфигурации Ansible»

Выполнил:

Якшин Артемий

Группа БР1.1

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

1. Разработать и настроить конфигурацию Ansible для автоматизации настройки виртуальных машин.
2. Написать Ansible-плейбуки для автоматической установки системных пакетов, настройки окружения и запуска сервисов.

Ход работы

1. Целевая машина и подход

Конфигурация применяется к тому же VPS, что и в ЛР4 (Ubuntu 24.04, 4 vCPU, 8 ГБ). Ansible работает без агента: управляющий узел — рабочий ноутбук (ansible-core 2.21.5, коллекции `community.docker` 5.4, `ansible.posix`), подключение по SSH, на сервере нужен только Python 3, а он есть в Ubuntu.

Сервер общий: на нём работает другой проект, поэтому на плейбук наложены жёсткие правила.

Правило	**Как соблюдается**
Docker, sshd и ufw не перенастраиваются	роль `preflight` только **проверяет** их (assert), Docker ставится лишь на чистой машине, где его нет; задач `restarted` для системных служб нет
пакеты не обновляются	`apt state: present` (не `latest`), индекс обновляется не чаще раза в сутки (`cache_valid_time`)
трогаем только своё	`/opt/rb-sem2`, пользователь `rb-deploy`, `/etc/sudoers.d/rb-deploy`, docker-ресурсы `rb-*`
ничего не сломать опечаткой	sudoers проверяется `visudo -cf` **до** установки; сначала прогон `--check --diff`
не мешать конвейеру CI	Ansible отвечает за состояние хоста, CI (ДЗ6) — за версию релиза. Плейбук не меняет `release.env` и checkout исходников, только гарантирует, что стенд запущен в текущем релизе

Адрес сервера в репозитории не хранится: инвентарь ссылается на SSH-алиас из `~/.ssh/config`.

2. Структура проекта

```
ansible/
├── ansible.cfg                  # инвентарь, роли, pipelining, профилирование задач
```

```
├── inventory/
│   ├── hosts.yml               # группа rb_servers: rb-server (ansible_host = SSH-
алиас)
│   └── group_vars/rb_servers.yml# параметры стенда: каталоги, cpuset, порт, пакеты,
ключ CI
├── site.yml                    # главный плейбук: роли по порядку, у каждой свой
тег
└── roles/
    ├── preflight/   # проверки: ОС, vCPU, Docker жив, порт 8088 свободен или наш
    ├── base/        # системные пакеты (git, curl, jq, openssl, rsync, ca-
certificates)
    ├── docker/      # Docker Engine + Compose + Buildx из официального репозитория
(только на чистой ВМ)
    ├── stand/       # каталоги, docker-сеть rb-edge, ограниченный сборщик,
исходники, секреты
    ├── deploy_access/ # пользователь CI, скрипты деплоя, sudoers, ключ с forced
command (ДЗ6)
    ├── edge/        # nginx — обратный прокси (ЛР4): конфиг, compose, reload по
handler'y
    └── app/         # compose-стенд приложения: запущен, здоров, отвечает через
nginx
```

В следующих работах в этот же Ansible-проект добавлены роли `monitoring` (ЛР6), `k3d` (ЛР7) и `k8s_monitoring` (ЛР8). В этом отчёте описано состояние на момент ЛР5.

`ansible.cfg`:

```
[defaults]
inventory = inventory/hosts.yml
roles_path = roles
interpreter_python = /usr/bin/python3
result_format = yaml
callbacks_enabled = ansible.posix.profile_tasks   # время каждой задачи
inject_facts_as_vars = False                       # факты только через
ansible_facts[...]
[ssh_connection]
pipelining = True                                  # меньше SSH-раундов на задачу
```

`site.yml` подключает роли по порядку. У каждой роли свой тег, поэтому можно применить часть конфигурации, например `ansible-playbook site.yml --tags edge`:

```
- name: Restaurant Booking — подготовка и конфигурация сервера
  hosts: rb_servers
  become: true
  roles:
    - { role: preflight, tags: [always] }
    - { role: base, tags: [base] }
    - { role: docker, tags: [docker] }
    - { role: stand, tags: [stand] }
    - { role: deploy_access, tags: [deploy_access] }
```

```
    - { role: edge, tags: [edge] }
    - { role: app, tags: [app] }
```

3. Роли

Установка системных пакетов (`base`, `docker`)

```
- name: Индекс пакетов не старше суток
  ansible.builtin.apt: { update_cache: true, cache_valid_time: 86400 }

- name: Базовые пакеты установлены
  ansible.builtin.apt:
    name: "{{ rb_base_packages }}"
    state: present          # не latest: на общем сервере ничего не обновляем
    install_recommends: false
```

Роль `docker` ставит Docker Engine только если `preflight` не нашёл его на машине. Используются репозиторий `download.docker.com` в формате deb822 (`deb822_repository` с ключом `signed_by`), пакеты `docker-ce`, `containerd.io`, `docker-buildx-plugin` и `docker-compose-plugin`, после установки служба включается. На сервере стенда Docker 29.8.1 уже стоит, поэтому роль лишь проверяет, что доступны `docker compose` и `docker buildx`.

Настройка окружения (`stand`, `deploy_access`)

- каталоги `/opt/rb-sem2/{app,bin,edge}` с правами из ЛР4/ДЗ6 (корень `0711`, `app/` — `0750`);
- docker-сеть `rb-edge` (`community.docker.docker_network`);
- сборщик `rb-builder` с лимитами. Сначала выполняется проверка `docker buildx inspect`, создание идёт только если сборщика нет;
- исходники: `ansible.builtin.git` с `update: false`. Репозиторий клонируется один раз, дальше нужный коммит выбирает CI;
- секреты: шаблон `app.env.j2` со случайными значениями (`community.general.random_string`) и `force: false`. Файл создаётся один раз с правами `0600` и при повторных запусках не перезаписывается, иначе инвалидировались бы JWT пользователей;
- доступ CI: пользователь `rb-deploy` с заблокированным паролем, скрипты `deploy.sh` и `deploy-gate.sh` (владелец root), sudoers с валидацией и ключ `authorized_key` с `key_options: restrict,command="…/deploy-gate.sh"` и `exclusive: true`:

```
- name: Sudo-правило только на deploy.sh (проверка синтаксиса visudo перед
установкой)
  ansible.builtin.copy:
    content: "rb-deploy ALL=(root) NOPASSWD: {{ rb_root }}/bin/deploy.sh\n"
    dest: /etc/sudoers.d/rb-deploy
    mode: "0440"
    validate: visudo -cf %s

- name: Ключ CI — только forced command, без pty и пробросов
  ansible.posix.authorized_key:
    user: rb-deploy
    key: "{{ rb_ci_deploy_pubkey }}"
    key_options: 'restrict,command="{{ rb_root }}/bin/deploy-gate.sh"'
    exclusive: true
```

Запуск сервисов (`edge`, `app`)

```
# roles/edge/tasks/main.yml
- name: Конфигурация и compose-файл edge-nginx
  ansible.builtin.copy:
    src: "{{ rb_local }}/labs/lab4/deploy/edge/{{ item }}"
    dest: "{{ rb_root }}/edge/{{ item }}"
    mode: "0644"
  loop: [nginx.conf, docker-compose.yml]
  notify: Reload edge nginx          # handler: nginx -t && nginx -s reload

- name: Edge-nginx запущен
  community.docker.docker_compose_v2:
    project_src: "{{ rb_root }}/edge"
    state: present
    wait: true

# roles/app/tasks/main.yml
- name: Стенд rb-app запущен и здоров
  community.docker.docker_compose_v2:
    project_src: "{{ rb_app_dir }}"
    files: [docker-compose.prod.yml]
    env_files: ["{{ rb_root }}/app/.env", "{{ rb_root }}/app/release.env"]
    state: present
    pull: missing
    build: never
    wait: true            # docker compose up --wait: ждём healthy всех контейнеров
    wait_timeout: 240

- name: Проверка через nginx
  ansible.builtin.uri: { url: "http://127.0.0.1:{{ rb_edge_port }}/health",
return_content: true }
  register: app_health
  retries: 10
  delay: 3
  until: app_health.status == 200
```

Конфигурация nginx берётся из каталога ЛР4, скрипты деплоя — из Д36. Ansible ничего не дублирует, а применяет уже версионированные файлы.

4. Сухой прогон на сервере

Сначала плейбук запущен в режиме `--check --diff`: Ansible показывает, что изменил бы, но ничего не меняет.

```
$ ansible-playbook site.yml --check --diff
TASK [preflight : Сводка]
ok: [rb-server] => msg: ["OS: Ubuntu 24.04", "vCPU: 4, RAM: 7941 MB", "Docker:
29.8.1", "Port 8088: ours"]
TASK [base : Базовые пакеты установлены] ............................... ok
TASK [stand : Общая docker-сеть rb-edge] ............................... ok
TASK [stand : Секреты стенда (генерируются один раз на сервере, ...)] .. ok
TASK [deploy_access : Ключ CI — только forced command, ...] ............ ok
TASK [edge : Edge-nginx запущен] ....................................... ok
TASK [app : Стенд rb-app запущен и здоров] ............................. ok
PLAY RECAP
rb-server : ok=25  changed=0  unreachable=0  failed=0  skipped=6
```

`changed=0` означает, что описанная в ролях конфигурация в точности совпадает с той, что была настроена вручную в ЛР4 и ДЗ6. Плейбук стал её формальным описанием.

5. Приведение к заданному состоянию (дрейф конфигурации)

Чтобы проверить, что плейбук действительно настраивает сервер, на нём вручную внесён «дрейф». Менялись только ресурсы стенда:

```
$ docker buildx rm rb-builder                       # удалён сборщик образов
$ rm -f /etc/sudoers.d/rb-deploy                    # CI потерял право на deploy.sh
$ echo "# manual drift" >> /opt/rb-sem2/edge/nginx.conf
$ cd /opt/rb-sem2/edge && docker compose stop       # остановлен nginx
$ curl -s -o /dev/null -w "%{http_code}" http://127.0.0.1:8088/health
000                                                 # стенд недоступен
```

Применение плейбука (`--diff` показывает, что именно исправлено):

```
$ ansible-playbook site.yml --diff
TASK [stand : Сборщик образов с лимитами (1 CPU, RAM) на ядрах 2,3]
changed: [rb-server]
TASK [deploy_access : Sudo-правило только на deploy.sh ...]
--- before
+++ after: /etc/sudoers.d/rb-deploy
@@ -0,0 +1 @@
+rb-deploy ALL=(root) NOPASSWD: /opt/rb-sem2/bin/deploy.sh
changed: [rb-server]
TASK [edge : Конфигурация и compose-файл edge-nginx]
--- before: /opt/rb-sem2/edge/nginx.conf
+++ after: БР1.1/Якшин Артемий/labs/lab4/deploy/edge/nginx.conf
@@ -53,4 +53,3 @@
     }
   }
```

```
 }
-# manual drift
changed: [rb-server] => (item=nginx.conf)
TASK [edge : Edge-nginx запущен]
changed: [rb-server]
TASK [app : Текущий релиз]
ok: [rb-server] => msg: {"service": "api-gateway", "status": "ok",
                         "version": "652a670235b7a27f1fb1db791cd601c2481821b5"}
RUNNING HANDLER [edge : Reload edge nginx]
changed: [rb-server]
PLAY RECAP
rb-server : ok=27  changed=5  unreachable=0  failed=0  skipped=5      (46.5 c)
```

Сборщик пересоздан, правило sudo возвращено после проверки `visudo`, ручная правка конфига nginx откатана, nginx запущен и перечитал конфиг через handler. Стенд снова отвечает, причём в том же релизе, который выкатил CI (`version` не изменился). Затем доступ CI проверен ключом конвейера: команда снова доходит до `deploy.sh` через sudo (`[app 0000000] commit is not on origin/main -> abort` — отказ уже от самого `deploy.sh`, а не от sudo).

6. Идемпотентность и качество

Повторный запуск сразу после исправления ничего не меняет:

```
$ ansible-playbook site.yml
PLAY RECAP
rb-server : ok=25  changed=0  unreachable=0  failed=0  skipped=6      (25.8 c)
```

Плейбук проверен `ansible-lint` 26.10 и проходит самый строгий профиль `production`:

```
$ ansible-lint site.yml
Passed: 0 failure(s), 0 warning(s) in 9 files processed of 9 encountered.
Last profile that met the validation criteria was 'production'.
```

По замечаниям линтера исправлены имена задач, добавлен `changed_when` для command-задач, устаревший `apt_repository` заменён на `deb822_repository`, факты читаются через `ansible_facts[...]`. После этого исчезли предупреждения ansible-core о будущей несовместимости.

Как запустить

```
cd labs/lab5/ansible
ansible-galaxy collection install community.docker ansible.posix community.general
# если ставили не пакет ansible
ansible-playbook site.yml --check --diff     # что изменится
```

```
ansible-playbook site.yml                       # применить
ansible-playbook site.yml --tags edge,app       # только nginx и приложение
```

Вывод

Разработана Ansible-конфигурация сервера стенда: инвентарь с групповыми переменными, главный плейбук и семь ролей. Они устанавливают системные пакеты и Docker, настраивают окружение (каталоги, сеть, ограниченный сборщик образов, секреты, пользователя CI с минимальными правами) и запускают сервисы (nginx и compose-стенд приложения) с проверкой здоровья. Плейбук учитывает, что сервер общий: системные службы только проверяются, пакеты не обновляются, изменения касаются лишь ресурсов стенда, а ответственность разделена между Ansible (состояние хоста) и CI (версия релиза). Сухой прогон показал, что ручная настройка из ЛР4/ДЗ6 полностью описана кодом. Эксперимент с дрейфом показал, что плейбук возвращает сервер в заданное состояние, а повторный запуск подтвердил идемпотентность (`changed=0`). Качество кода подтверждено `ansible-lint` в профиле `production`.